

Problem solving through State Space Search

(Building Goal-Based Agents)

Water Jug Problem in AI

- The water Jug Problem, as the name suggests, is a problem where two jugs of water are given, say one is a 4-litre one, and the other one is a 3-litre one, but none of the measuring markers is mentioned on any of it. There is a pump available to fill the jugs with water. How can you exactly pour 2 liters of water into a 4-litre jug? Assuming that both the jugs are empty, the task is to find a solution to pour 2-litre water into a 4-litre jug.

A Solution

- Start state (0,0)
- (0,3) Fill the 3-litre Jug
- (3,0) Pour all the water from a 3-litre jug into a 4-litre jug
- (3,3) Fill the 3-litre Jug
- (4,2) Pour water from a 3-litre jug into a 4-litre jug until it is full
- (0,2) Empty 4-litre jug on the ground
- (2,0) Pour all the water from a 3-litre jug into a 4-litre jug

8-Puzzle

Given an initial configuration of 8 numbered tiles on a 3 x 3 board, move the tiles in such a way so as to produce a desired goal configuration of the tiles.

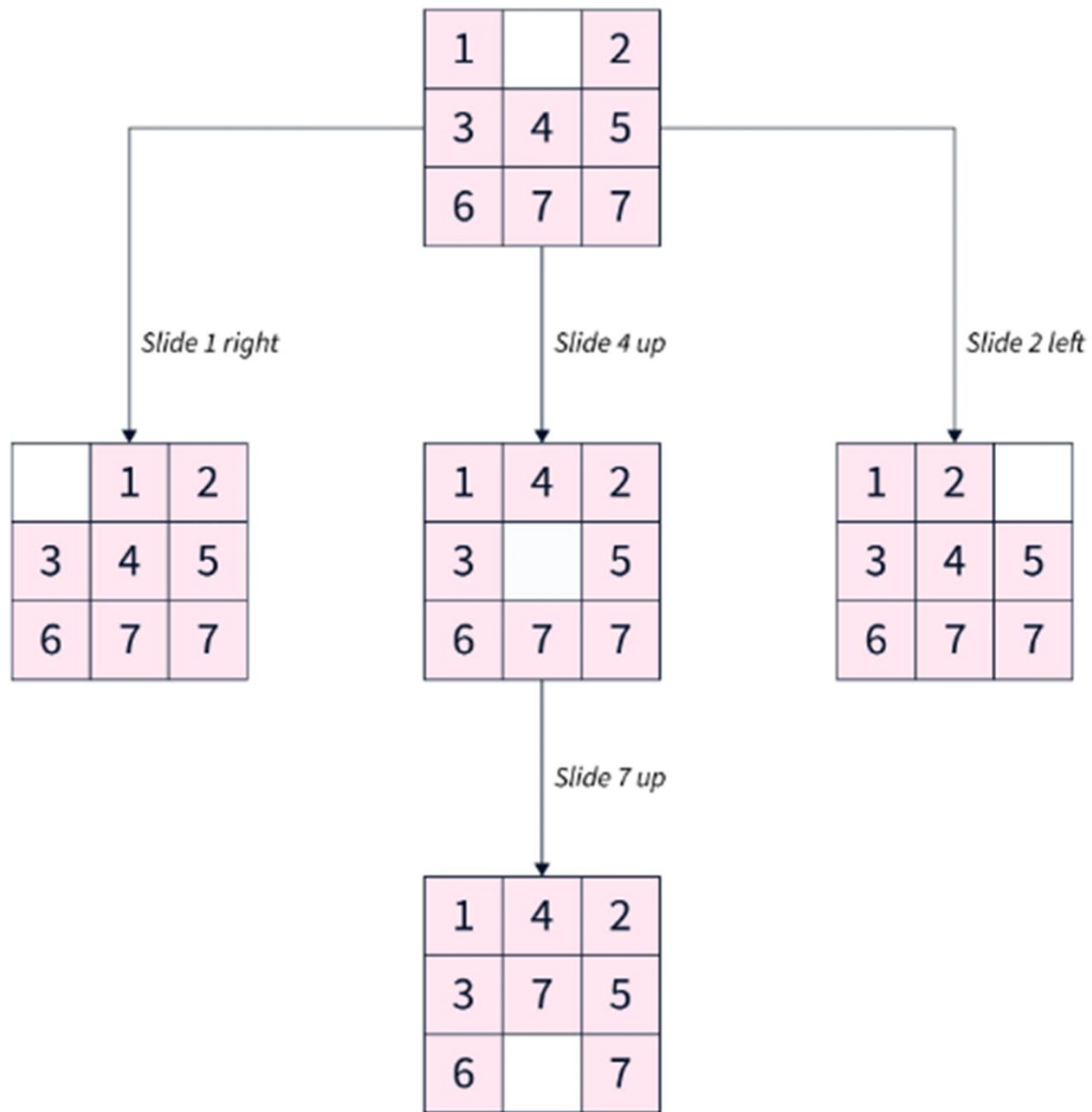
1		2
3	4	5
6	7	7

Current State

1	4	2
3	7	5
6		7

Target State

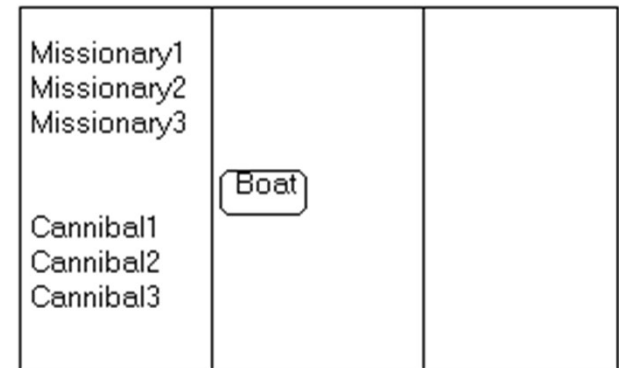
Solution



Missionaries and Cannibals

There are 3 missionaries, 3 cannibals, and 1 boat that can carry up to two people on one side of a river.

- **Goal:** Move all the missionaries and cannibals across the river.
- **Constraint:** Missionaries can never be outnumbered by cannibals on either side of river, or else the missionaries are killed.
- **State:** configuration of missionaries and cannibals and boat on each side of river.
- **Operators:** Move boat containing 1 or 2 occupants across the river (in either direction) to the other side.



3 Missionaries and 3 Cannibals wish to cross the river. They have a boat that will carry two people. Everyone can navigate the boat. If at any time the Cannibals outnumber the missionaries on either bank of the river, they will eat the Missionaries. Find the smallest number of crossings that will allow everyone to cross the river safely.

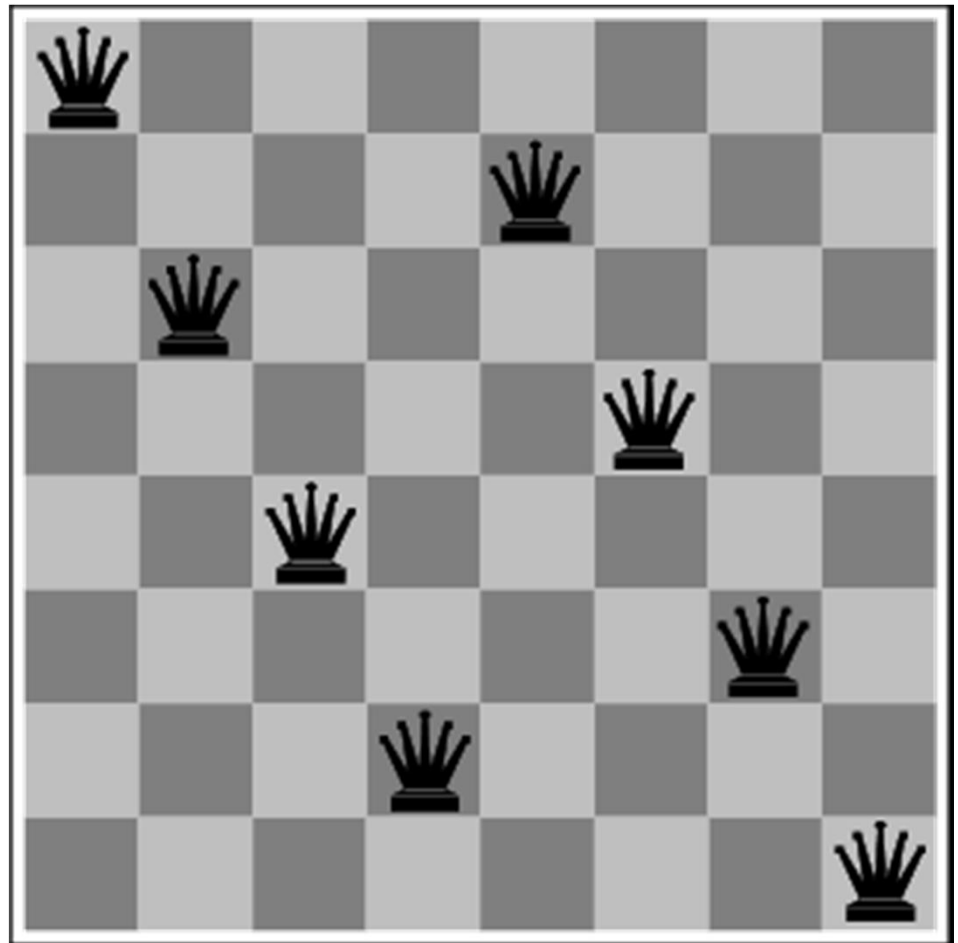
The problem can be solved in 11 moves. But people rarely get the optimal solution, because the MC problem contains a 'tricky' state at the end, where two people move back across the river.

Missionaries and Cannibals Solution

	<u><i>Near side</i></u>		<u><i>Far side</i></u>	
0 Initial setup:	MMMCCC	B	-	
1 Two cannibals cross over:	MMMC		B	CC
2 One comes back:	MMMCC	B	C	
3 Two cannibals go over again:	MMM		B	CCC
4 One comes back:	MMMC	B	CC	
5 Two missionaries cross:	MC		B	MMCC
6 A missionary & cannibal return:	MMCC	B	MC	
7 Two missionaries cross again:	CC		B	MMMC
8 A cannibal returns:	CCC	B	MMM	
9 Two cannibals cross:	C		B	MMMCC
10 One returns:	CC	B	MMMC	
11 And brings over the third:	-		B	MMMCCC

The 8-Queens Problem

**Place eight queens
on a chessboard
such that no
queen attacks any
other!**



Summary

- **State:** A specific configuration of the problem.
- **State Space:** Collection possible states
- **Initial State:** The starting point of the search.
- **Goal State:** The desired end configuration.
- **Transition:** An **action (Operator/Action)** that changes one state to another.
- **Path:** A sequence of states connected by transitions.
- **Search Strategy:** The method used to explore the state space.

Formalizing Search in a State Space

- A state space is a **graph**, (V, E) where V is a set of **nodes** and E is a set of **arcs**, where each arc is directed from a node to another node due to an action executed.
- **node**: corresponds to a **state**
 - state description
 - plus optionally other information related to the parent of the node, operation to generate the node from that parent, and other bookkeeping data)
- **arc**: corresponds to an applicable action/operation.
 - the source and destination nodes are called as **parent (immediate predecessor)** and **child (immediate successor)** nodes with respect to each other
 - ancestors(predecessors) and descendents (successors)
 - each arc has a *fixed, non-negative* **cost** associated with it, corresponding to the cost of the action

- **node generation:** making explicit a node by applying an action to another node which has been made explicit
- **node expansion:** generating **all** children of an explicit node by applying **all** applicable operations to that node
- One or more nodes are designated as **start nodes**
- A **goal test** predicate is applied to a node to determine if its associated state is a goal state
- A **solution** is a sequence of operations that is associated with a path in a state space from a start node to a goal node
- The **cost of a solution** is the sum of the arc costs on the solution path

- **State-space search** is the process of searching through a state space for a solution by making explicit a sufficient portion of an implicit state-space graph to include a goal node.
 - Hence, initially $V=\{S\}$, where S is the start node; when S is expanded, its successors are generated and those nodes are added to V and the associated arcs are added to E . This process continues until a goal node is generated (included in V) and identified (by goal test)
- During search, a node can be in one of the three categories:
 - Not generated yet (has not been made explicit yet)
 - **OPEN**: generated but not expanded
 - **CLOSED**: expanded
 - Search strategies differ mainly on **how to select an OPEN node** for expansion at each step of search

A General State-Space Search Algorithm

- **Node n**
 - state description
 - parent (may use a backpointer) (if needed)
 - Operator used to generate n (optional)
 - Depth of n (optional)
 - Path cost from S to n (if available)
- **OPEN** list
 - initialization: {S}
 - node insertion/removal depends on specific search strategy
- **CLOSED** list
 - initialization: {}
 - organized by backpointers

A General State-Space Search Algorithm

open := {S}; closed := {};

repeat

 n := *select*(open); /* select one node from open for expansion */
 */

if n is a goal

then exit with success; /* delayed goal testing */

expand(n)

 /* generate all children of n

 put these newly generated nodes in open (check duplicates)

 put n in closed (check duplicates) */

until open = {};

exit with failure